

OAT 2

Web API Avançada da MecâniQA

Modelagem de Domínio, Padrão Builder e Entidade Associativa

Autores:

Alex Gabriel Alves dos Santos

Gustavo Azevedo Diniz

Gustavo Muniz dos Anjos

João Paulo Rosa Batista

Matheus Silva do Monte

Agenda

Objetivo da Apresentação

Apresentar a evolução da arquitetura da MecâniQA Web API na OAT 2, com foco na introdução de Ordens de Serviço, Pedidos de Peças, resolução relacional via Entidade Associativa, aplicação do padrão Builder com encapsulamento rigoroso e a visão lógica do Controller no fluxo de criação de pedidos.

1. Domínio da OAT 2

Novas entidades OrdemServico e PedidoPecas, enums de status e regras de ciclo de vida operacional.

2. Padrão Builder Manual

Construção fluente de objetos complexos, proteção de invariantes e construtores estritamente privados.

3. Entidade Associativa

Modelagem de ItemPedidoPecas para desacoplamento N:N, suporte a itens repetidos e histórico de preços.

4. Diagrama de Classes OAT 2

Atualização consolidada com entidades, builders manuais, DTOs mapeados e repositórios.

5. Diagrama de Atividades (US03)

Visão lógica do Controller no endpoint POST /api/pedidos com validações e respostas HTTP.

Novas entidades estruturam o fluxo operacional

Agregados de Negócio Ordem de Serviço (OS) & Pedidos

A fase 2 expande a oficina para cobrir o fluxo completo de atendimento ao cliente:

- **OrdemServico:** Agregado central que reúne dados do cliente, identificação do veículo, lista de `Servico` executados e vínculo com `PedidoPecas`.
- **PedidoPecas:** Gerencia o fornecimento e consumo de insumos na oficina, totalizando peças e controlando o orçamento.
- **Recálculo Automático:** O método `recalcularValorTotal()` soma dinamicamente a mão de obra tabelada e o total de peças, garantindo consistência em qualquer alteração.

Controle de Estados Ciclo de Vida por Enums Ricos

Transições operacionais estritas mapeadas em enums com descrições semânticas:

- **StatusOrdemServico:**
 - `ABERTO` ("Aberto")
 - `PENDENTE_PAGAMENTO` ("Pendente de Pagamento")
 - `PAGO` ("Pago")
 - `EM_EXECUCAO` ("Em Execução")
 - `EXECUTADO` ("Executado")
- **StatusPedidoPecas:**
 - `ORCANDO` ("Orçando") - Status inicial obrigatório
 - `PENDENTE_PAGAMENTO` ("Pendente de Pagamento")
 - `PAGO_FATURADO` ("Pago/Faturado")
 - `ENTREGUE` ("Entregue")

Padrão Builder manual assegura integridade

Estrutura Técnica Implementação Manual Pura

O Builder foi construído como classe estática aninhada sem frameworks externos:

```
// Construtor privado blinda instanciação arbitrária
private OrdemServico(Builder builder) {
    this.codigo = builder.codigo;
    this.descricao = builder.descricao;
    this.cliente = builder.cliente;
    this.veiculo = builder.veiculo;
    this.status = builder.status != null ?
        builder.status : StatusOrdemServico.ABERTO;
    this.servicos = builder.servicos;
    this.pedidoPecas = builder.pedidoPecas;
    this.valorTotal = calcularValorTotal(...);
}

public static Builder builder() {
    return new Builder();
}
```

Vantagens Arquiteturais Benefícios no Modelo de Domínio

- **Construtores Estritamente Privados:** Impede chamadas acidentais a construtores com muitos parâmetros ou criação de entidades em estado incompleto. Testes com reflexão asseguram essa restrição.
- **Interface Fluente:** Métodos concisos e autoexplicativos como `.servico(...)` e `.item(...)` simplificam a composição progressiva das listas.
- **Valores Padrão e Consistência:** Garantia de que coleções iniciem vazias em vez de nulas e que datas de auditoria e totais financeiros sejam inicializados corretamente no `build()`.

Entidade Associativa resolve relação N:N

Modelagem de ItemPedidoPecas Atributos e Comportamento Próprio

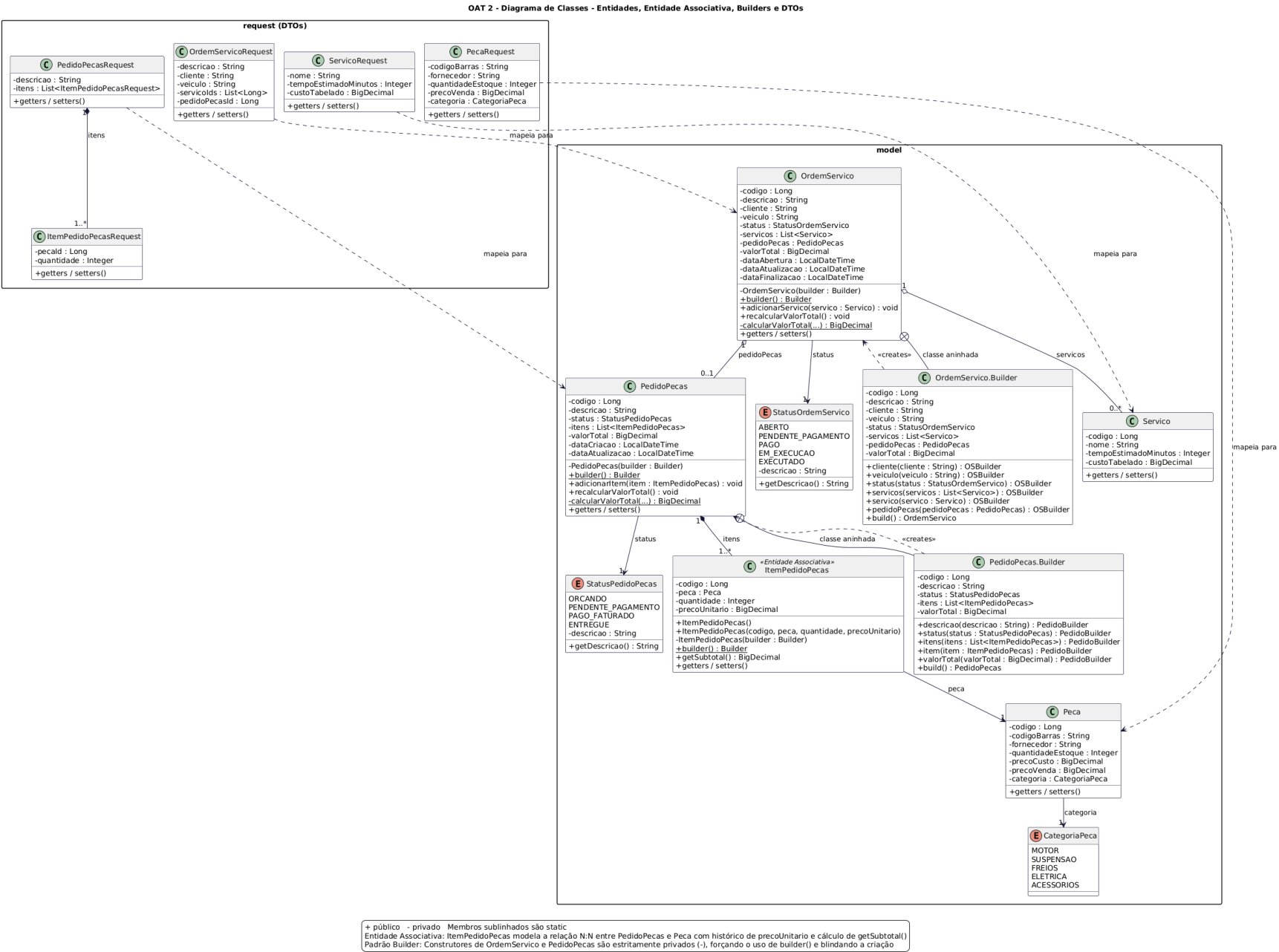
A entidade associativa conecta `PedidoPecas` a `Peca` carregando o estado específico da associação:

- `codigo: Long` - Identificador próprio do item no pedido.
- `peca: Peca` - Referência direta à peça do catálogo.
- `quantidade: Integer` - Quantidade requisitada.
- `precoUnitario: BigDecimal` - Preço de venda congelado no momento da inclusão.
- `getSubtotal(): BigDecimal` - Cálculo seguro multiplicando quantidade por preço unitário.

Garantias do Negócio Por que usar Entidade Associativa?

- **Itens Duplicados ou em Lotes:** Permite que a mesma peça seja adicionada múltiplas vezes no mesmo pedido (ex.: pastilhas dianteiras e traseiras com embalagens separadas) sem violar unicidade de chaves.
- **Imunidade a Flutuações de Mercado:** O preço unitário do item é capturado na data da criação do pedido. Se o preço da peça for alterado no catálogo posteriormente, o valor do pedido original permanece intacto.
- **Precisão Financeira:** O uso de `BigDecimal` elimina erros de arredondamento em cálculos fiscais e orçamentários.

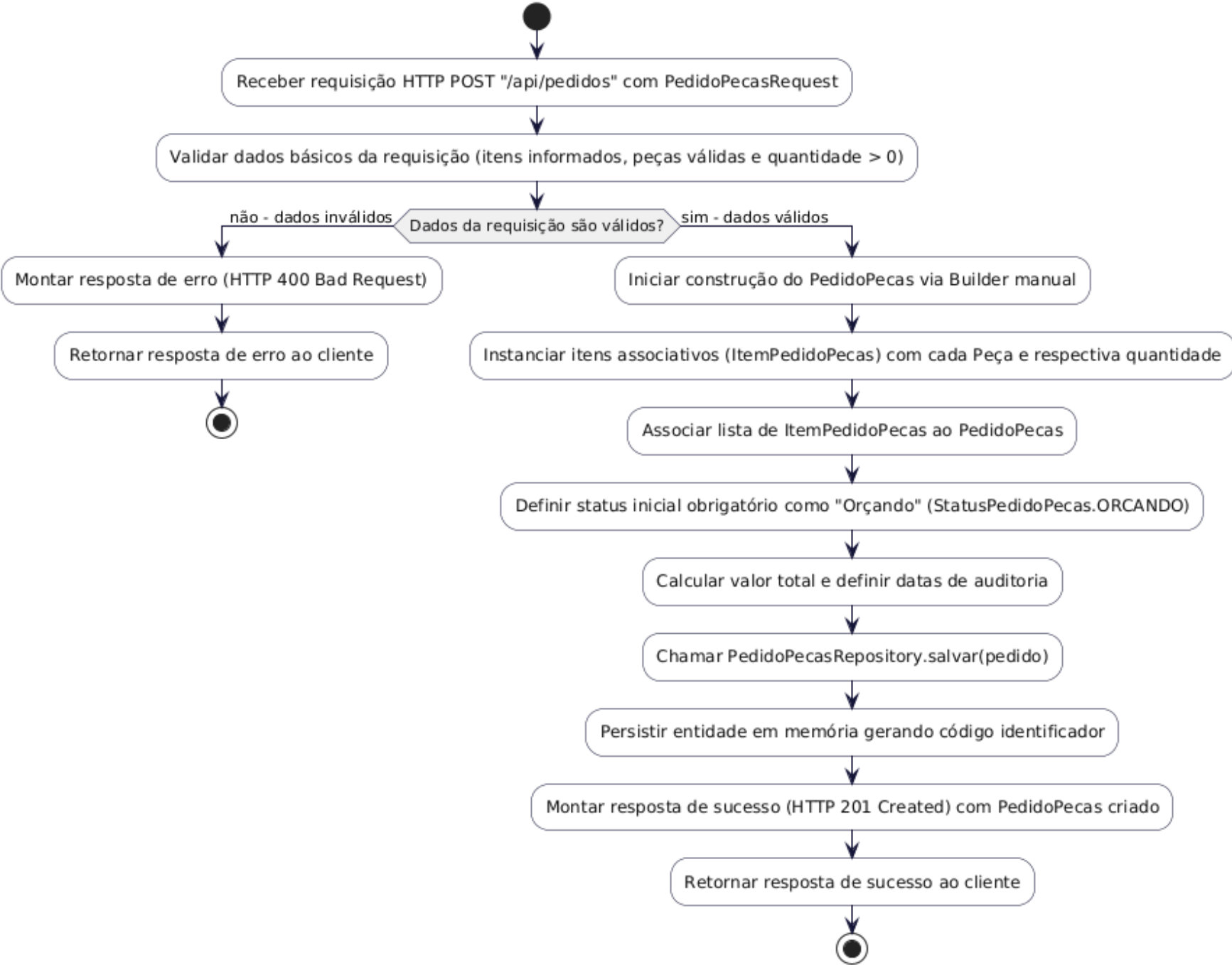
Diagrama de Classes atualizado com OAT 2



Elementos Atualizados: Entidades de domínio (`OrdemServico` , `PedidoPecas`), Entidade Associativa (`ItemPedidoPecas`), Builders fluentes com construtores protegidos, enums de status enriquecidos e pacote de DTOs desacoplados (`PedidoPecasRequest` , `ItemPedidoPecasRequest` , `OrdemServicoRequest`).

Diagrama de Atividade: US03 - Criar Pedido (POST /api/pedidos)

US03 - Criar Pedido de Peças - Diagrama de Atividades (Visão do Controller)



Visão Lógica do Controller

Fluxo Decisório do Endpoint

- **1. Recepção do Payload:** O Controller recebe requisição `POST /api/pedidos` com o DTO `PedidoPecasRequest`.
- **2. Validação Defensiva:** Verifica integridade da lista de itens, existência das peças solicitadas e `quantidade > 0`.
- **3. Fluxo Alternativo (Erro):** Caso os dados sejam inválidos, monta e retorna imediatamente `HTTP 400 Bad Request`.
- **4. Fluxo Principal (Sucesso):**
 - Instancia itens via `ItemPedidoPecas.builder()`
 - Constrói o pedido com `StatusPedidoPecas.ORCANDO`
 - Delega persistência ao repositório Singleton
 - Emite resposta `HTTP 201 Created` com o recurso criado.

Síntese da OAT 2 Maturidade da Arquitetura

- **Consistência Estrutural:** O domínio agora suporta o ciclo completo da oficina, combinando mão de obra tabelada e controle de materiais em pedidos de peças.
- **Encapsulamento Robusto:** Eliminação de estado transitório ou objetos mal inicializados através de Builders estritos com construtores estritamente privados.
- **Desacoplamento por Camadas:** DTOs de entrada isolam a API HTTP do domínio de entidades, enquanto diagramas de atividades padronizam as decisões dos controllers.

Fontes e Artefatos Referências do Projeto

- **Repositório do Projeto:**
 - `Guztaver/mecaniQA-api-rio-de-janeiro`
- **Documentação e Guias:**
 - Guia de Desenvolvimento Web Orientado a Objetos - UNEX 2026
 - OAT 1 - Web API Fundacional da MecâniQA
 - Modelo de Apresentação Institucional UNEX
- **Suíte de Testes Automatizados:**
 - `OrdemServicoTest` (verificação de construtores privados e recálculo)
 - `PedidoPecasTest` (entidade associativa e multiplicidade de itens)